All Proof of Work But No Proof of Play

Hayder Tirmazi

City College of New York
hayder.research@gmail.com

1 Introduction

Speedrunning is a competition that emerged from communities of early video games such as Doom (1993) [8]. Speedrunners try to finish a game in minimal time [14]. Provably verifying the authenticity of submitted speedruns is an open problem. Traditionally, best-effort speedrun verification is conducted by on-site human observers, forensic audio analysis [15], or a rigorous mathematical analysis of the game mechanics[1]. Such methods are tedious, fallible, and, perhaps worst of all, not cryptographic. Motivated by naivety and the Dunning-Kruger effect, we attempt to build a system that cryptographically proves the authenticity of speedruns. This paper describes our attempted solutions and ways to circumvent them. Through a narration of our failures, we attempt to demonstrate the difficulty of authenticating live and interactive human input in untrusted environments, as well as the limits of signature schemes, game integrity, and provable play.

Running Example. We will use an open-source video game [12] we developed as a running example to both demonstrate techniques for generating fraudulent speedruns and our attempts at cryptographically verifying speed run authenticity. Our video game, lilypond, is developed in roughly 7K lines of `C` code. We have also released a web version of lilypond that can be played by readers directly on their web browser [13]. The game has a playable character called Mano who can move left and right, jump, sprint, and defeat enemies by jumping on top of them, i.e, *squishing* them. Players control Mano's movements using the keyboard. Mano can also collect coins by colliding with them. If Mano collides with an enemy, such as a bug or a ghost, she is relocated back to the start of the level. If Mano falls into water or lava, she is also relocated to the start.

Notation. We introduce the notation we will be using throughout the paper. $\mathcal{A}$ denotes a PPT adversary. $\sigma_t \in \Sigma$ denotes the internal state of the game at time t, where Σ is the set of all such states. In Lilypond, this is the struct containing Mano's co-ordinators, current position, etc., and a level struct containing similar information for every other object in the game(Lst. 1.1). $I = \mathbb{N} \times \mathcal{K} = \{(t_i, k_i)\}$ is a set of timestamped keystrokes, with t_i being the frame number, and $\mathcal{K}$

[1] For example, the 29-page official report [6] for the Dream Minecraft case [10] rigorously proves that the probability of the investigated speed run *not* being fraudulent is $\leq 1.326 \times 10^{-13}$, with probabilities taken over the random coins used in item bartering and other game mechanics.

being the set of playable keys in the game (up, down, jump, $\cdots$). I models the interactive inputs received by the game. Pixel depth n_p is the number of bits used to encode a pixel on a screen. Using $P = \{0,1\}^{n_p}$, a screenshot $s \in P^{wh}$ models a single rendering of the game on a screen of $w \times h$ pixels (each $p \in P$ is a single pixel on the screen). A screenshotting function $f : \Sigma \mapsto P^{wh}$ maps a game state to a screenshot. A speedrun $S \subset \mathbb{N} \times P^{wh} = \{(t_i, s_i)\}$ is a set of timestamped screenshots s_i. Now we can use our notation to formally define a video game.

Definition 1. A PPT algorithm G is a video game if it interactively takes a set of timestamped keystrokes I and outputs a set of states $\mathcal{S} = \{\sigma_0, \cdots, \sigma_n\}$ using a transition function $T : \mathbb{N} \times \mathbb{N} \times \mathcal{K} \times \Sigma \mapsto \Sigma$ such that $\sigma_{t+1} \leftarrow T(t_s, t, \sigma_t, k_t)$. Note that t_s is the system time while t is the in-game (logical) time.

Fraud Techniques. We discuss four common methods of speedrun fraud. The most common method is splicing [4]. In a lilypond run (Fig. 1) with a jump, an optimal speedrun involves Mano clearing the jump in one attempt. $\mathcal{A}$ can make several failed attempts at the jump (falling into the water) and cut that footage out of the final speedrun recording. $\mathcal{A}$ can also stitch together only the successful attempts for all of Mano's jumps in multiple parts of the game to create a single recording in which Mano appears to clear all jumps successfully in one run.

The second technique is logic modification either in the game binary or at runtime. $\mathcal{A}$ can modify the game binaries to change the speed at which Mano falls to the ground or Mano's jump height (Lst. 1.2). $\mathcal{A}$ can also modify the in-memory game state [11] at runtime (Lst. 1.1) to increase the value of coins collected by Mano or the level Mano is in currently.

The third technique is simulated input. $\mathcal{A}$ can simulate keyboard input [15] to send a pre-recorded set I of input values with frame-perfect precision. I may be either algorithmically generated or a replay of a more successful speedrunner.

The fourth technique is timestamp skew. $\mathcal{A}$ can slow down in-game timers (Lst. 1.3) that control Mano's acceleration and other properties, or the system timers that the in-game timers synchronize on. $\mathcal{A}$ then plays a slowed down version of the game which makes the game significantly easier. $\mathcal{A}$ speeds up the final speedrun recording so it looks like the game was played at its original pace.

Execution environments. There are two common execution environments for video games: thin clients [2, 3, 5] and thick clients [5]. Thin clients run the game binary on a cloud server, receive player input over the network, and stream visual output back to the user. The client on the player's device is *thin*, i.e, it only acts as an input collector and video streamer. Nvidia [7], Sony [9], and Amazon [1] all offer such cloud gaming subscriptions, allowing a game to be played entirely on their trusted hardware and the visual and audio output streamed to the player's client. A thick client runs the game binary on the player's device. It may use a cloud server for sending state updates in the case of multiplayer games, but does not execute the game directly on the server.

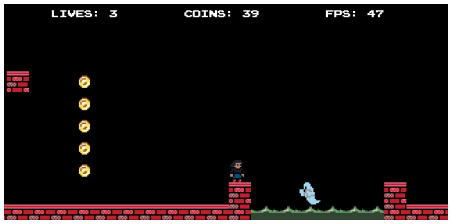

Fig. 1: Experimental game lilypond

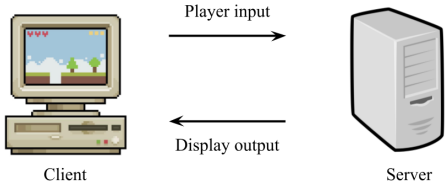

Fig. 2: Thin client environment

2 Security Notions

We first discuss game execution oracles. Next, we introduce two security games in increasing order of adversarial power. Finally, we introduce a security notion for the games.

Game Execution Oracles. A thin client oracle O_1 for a game G and screenshotting function f takes a timestamped keystroke (t, k) as input and outputs a screenshot $s = f(G(t, k))$. Thus $\mathcal{A}$ plays the game without having access to either the internal game state, the system timer, or the screenshotting function. A thick client oracle O_2 for a game G takes as input all the input values for G's transition function T, namely, the system time t_s, the game time t, a keystroke k_t, and a game state σ_t. O_2 outputs the game state $\sigma_{t+1} = T(t_s, t, \sigma_t, k_t)$. $\mathcal{A}$ chooses their own screenshotting function for converting the returned game state to a screenshot.

Security Games. We now use the game execution oracles to define two security games. For a video game G and $n, \lambda \in \mathbb{N}$ (λ is the security parameter), we define `ThinClientGame` using a thin client oracle O_1 as follows.

`ThinClientGame`$(\mathcal{A}, G, \mathsf{V}, t, \lambda)$:

1. $\mathcal{A}$ plays G using O_1 up to game time t to get screenshot s_t.
2. $\mathcal{A}$ outputs challenge screenshot s^*
3. If V accepts s^* and $s^* \neq s_t$, return 1 ($\mathcal{A}$ wins). Otherwise, return 0.

Similarly, we define `ThickClientGame` using a think client oracle O_2 and a screenshotting function f as follows.

`ThickClientGame`$(\mathcal{A}, G, \mathsf{V}, t, \lambda)$:

1. $\mathcal{A}$ plays G using O_2 up to game time t to get state σ_t.
2. $\mathcal{A}$ outputs challenge screenshot s^*
3. If V accepts s^* and $s^* \neq f(\sigma_t)$, return 1 ($\mathcal{A}$ wins). Otherwise, return 0.

We know define a security notion for both games.

Definition 2. Let a speedrun construction $C = (G, \mathsf{V}, f)$ be a 3-tuple with a game G and corresponding verifier V, and screenshotting function f. Let $\mathcal{A}$ be a

PPT adversary. We say C is an (t, ε)-secure construction under a security game `Game` if

$$\Pr[\texttt{Game}(\mathcal{A}, G, \mathsf{V}, t, \lambda) = 1] \leq \varepsilon$$

where λ is the security parameter and the probabilities are taken over the random coins of $\mathcal{A}$ and G.

Thin Client Security We show that any speedrun construction C can be transformed (in polynomial time) into a construction C' that is secure under `ThinClientGame` as long as we are allowed to modify least $\frac{n_p}{\lambda}$ pixels in every screenshot. Recall that n_p is the number of bits per pixel and λ is the security parameter.

Theorem 1. Let $C = (G, V, f)$ be a speedrun construction. Assuming one-way functions, there exists a modified construction C' that is $(t, \mathsf{negl}(\lambda))$-secure for any $t \in \mathbb{N}$ that modifies at most $\frac{n_p}{\lambda}$ pixels in each screenshot s_i of the speedrun.

Proof Sketch. Encode signatures with timestamp, input, and game state into pixels of the screenshot, treating them as authenticated logs of play. V checks these signatures in the screenshot.

Proof. We modify G's transition function T to create a modified function T'. Let $\sigma_{t+1} = T(t_s, t, \sigma_t, k_t)$ and $y = t_s \| t \| \sigma_t \| k_t$. Instead of returning σ_{t+1} directly, T' returns $\sigma'_{t+1} = (\sigma_{t+1}, \mathsf{Sign}_{\mathsf{sk}}(y))$ where $\mathsf{Sign}_{\mathsf{sk}}$ is a digital signature scheme that outputs a signature of length λ bits using a private key sk of length $\mathcal{O}(\lambda)$ embedded in the game binary. We modify the screenshotting function f to create a new function f' that first invokes f and then writes the signature in $\frac{n_p}{\lambda}$ pixel indices chosen beforehand. Note that each screenshot has $w \times h \times n_p$ bits, so encoding the λ bits long signature means modifying $\frac{n_p}{\lambda}$ pixels. We modify the verifier V to only accept a screenshot if the values of the chosen pixel indices equal the signature value. Suppose $\mathcal{A}$ outputs a screenshot $s^* \neq s_t$ that V accepts. Then s^* must contain a valid signature on some $y' \neq y$, i.e., a forgery. Since we assume one-way functions exist, the unforgability of $\mathsf{Sign}_{\mathsf{sk}}$ implies that the probability of $\mathcal{A}$ winning the security game is negligible. □

Under a thin client, we are naturally immune to logic modification and timestamp skew as $\mathcal{A}$ does not have access to the game binaries, memory, or system timers. However, thin client games are still vulnerable to splicing and simulated inputs. `ThinClientGame` and its security notion does succeed in mitigating splicing, but it does not mitigate simulated input.

3 Epic Fails & Open Problems

We now discuss the failures and shortcomings of our security notions and our fruitless attempts at improving them. We leave some open problems for the community so brighter minds may succeed where we failed.

Problem 1: security notions covering simulated inputs. Our security games do not provide any provable guarantees against simulated inputs. We were unable to propose reasonable assumptions for what such a security notion would look like. One notion we thought about was computational indistinguishability. Challenger $\mathcal{C}$ generates an honest input I_0 and asks adversary $\mathcal{A}$ for a simulated input I_1. $\mathcal{C}$ then flips a bit b and gives $\mathcal{A}$ the input I_b. Can we bound $\mathcal{A}$'s probability of inferring b and, if so, does this provide meaningful security? We leave the formulation of a meaningful security notion that covers simulated inputs as an open problem.

Problem 2: empirical methods for simulated input detection. We tried experimentally detecting simulated inputs. We logged timestamped keystrokes received from a human speedrun of lilypond and edited the log to remove suboptimal inputs. For example, the honest speedrun walks Mano in the wrong direction (left key) before turning back. The edited log removes the left keystrokes for the missteps and the extra right keystrokes of the return journey, giving a dishonest speedrun where Mano moves right without the missteps, saving time. We were unable to distinguish this modified speedrun from a second honest speedrun, where the player simply took the correct path the first time.

Discounting trusted hardware, e.g, keys having a biometric fingerprint sensor, we did not find a way to detect a dishonest speedrun compared to repeated honest speedruns with the player organically performing better over time. For other metrics such variations in keypress interarrival time and the entropy of the set of input stream, we found no convincing arguments for why any such metric cannot also be efficiently simulated, especially from training data taken directly from honest speedruns. We leave this as an open problem.

Problem 3: provable security, or an impossibility proof, for the thick client setting. Our attempts at finding a secure construction for `ThickClientGame` failed. On a thick client, $\mathcal{A}$ controls everything, including the game binary, memory, system timers, and input hardware. We question whether meaningful security guarantees under such assumptions are even possible.

Problem 4: new execution environments and security games. While our security games are based on execution environments deployed today, they seem to be difficult to achieve guarantees for. `ThinClientGame` assumes adversary $\mathcal{A}$ controls only the input. We attempted an even stronger model where $\mathcal{A}$ cannot manipulate input streams, but that is akin to requiring a player to record their speedruns in Fort Knox. `ThickClientGame`, on the other hand, assumes $\mathcal{A}$ controls everything. We also tried making assumptions that were slightly stricter but still realistic, such as game binary signing, but with limited success. We leave this as an open problem.

While this paper may serve as proof of work, we were unable to provide a proof of play. We did make some friends along the way. Though they were just bots, running simulated inputs that are doomed to stay.

References

1. Amazon: Amazon Luna. https://luna.amazon.com/ (2025)
2. Chang, Y.C., Tseng, P.H., Chen, K.T., Lei, C.L.: Understanding the performance of thin-client gaming. In: 2011 IEEE International Workshop Technical Committee on Communications Quality and Reliability (CQR). pp. 1–6. IEEE (2011)
3. Claypool, M., Finkel, D., Grant, A., Solano, M.: Thin to win? network performance analysis of the onlive thin client game system. In: 2012 11th Annual Workshop on Network and Systems Support for Games (NetGames). pp. 1–6. IEEE (2012)
4. Frank, A.: Cheating in speedrunning is easier than you'd think. Polygon (2017)
5. Lee, K., Chu, D., Cuervo, E., Kopf, J., Degtyarev, Y., Grizan, S., Wolman, A., Flinn, J.: Outatime: Using speculation to enable low-latency continuous interaction for mobile cloud gaming. In: Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services. pp. 151–165 (2015)
6. Minecraft Speedrunning Team: Dream investigation results (official report) (2020)
7. Nvidia: Geforce NOW. https://www.nvidia.com/en-us/geforce-now/ (2025)
8. Paez, D.: Coined: How "speedrunning" became an Olympic-level gaming competition. Inverse Magazine (2020)
9. Playstation, S.: Playstation Portal. https://www.playstation.com/en-us/support/subscriptions/psportal-cloud-game-streaming/ (2025)
10. Ritchie, S.: Why Are Gamers So Much Better Than Scientists at Catching Fraud? The Atlantic (2021)
11. Source, O.: Cheat Engine. https://github.com/cheat-engine/cheat-engine (2025)
12. Tirmazi, H.: Github Source Code for lilypond. https://github.com/lilyfarm/lilypond (2025)
13. Tirmazi, H.: Playable web version of lilypond. https://lilyfarm.itch.io/lilypond (2025)
14. White, C.: The History of My $10,000 Speedrun Challenge. Youtube channel penguinz0 (2023)
15. Wright, S.: How the scourge of cheating is changing speedrunning. Arstechnica (2019)

A Code Listings

```
struct mano {
  double x, y, vx, vy;
  bool air, ladder, sprint, jump;
  unsigned int lives, coins;
  size_t respawn_x, respawn_y;
};

struct sprite {
  double x, y, vx, vy;
  bool removed;
  union sprite_data data;
};

struct level {
```

```
  struct sprite_type **passive_sprites;
  struct array *active_sprites;
};
```

Listing 1.1: Game State

```
static const Sint64 GRAVITY = 600;
static const Sint64 WALK = 72;
static const Sint64 SPRINT = 96;
static const Sint64 LADDER = 64;
static const Sint64 FALL_MAX = 128;
static const Sint64 JUMP = 256;
static const Sint64 SHORT_JUMP = 192;
```

Listing 1.2: Game Physics Constants

```
struct fps_timer {
  Uint64 delay;
  Uint64 time_left;
};

enum {
  FRAME_RATE = 48,
  FRAME_TIME = 1000 / FRAME_RATE,
  MIN_FRAME_RATE = 24,
  MAX_FRAME_TIME = 1000 / MIN_FRAME_RATE
};
```

Listing 1.3: Game Timers